Implementing Stack and Queue

1. Project Overview

The Stack and Queue Implementation project is a Python-based application developed to understand two fundamental linear data structures: Stack and Queue. The project demonstrates their implementation using different approaches and applies these data structures to practical problems such as balanced parentheses checking and task scheduling.

The project includes Stack implementation using a Python list and linked list, Queue implementation using a Python list and Circular Queue, along with practical applications of Stack and Queue.

2. Problem Statement

Develop Python programs to understand and implement Stack and Queue data structures using different approaches. The programs should demonstrate their basic operations and practical applications using Stack for balanced parentheses checking and Queue for task scheduling.

The project includes:

- Implementing a Stack using a Python list
- Implementing a Stack using a linked list
- Implementing a Queue using a Python list
- Implementing a Circular Queue
- Checking balanced parentheses using a Stack
- Implementing a task scheduler using a Queue

3. Features

Stack Operations

- Implement Stack using Python List

- Implement Stack using Linked List
- Perform push, pop, and peek operations
- Check whether the Stack is empty
- Follow the LIFO principle

Queue Operations

- Implement Queue using Python List
- Implement Circular Queue
- Perform enqueue, dequeue, and peek operations
- Check empty and full conditions
- Follow the FIFO principle

Practical Applications

- Check balanced parentheses using Stack
- Implement task scheduling using Queue
- Process tasks in FIFO order
- Validate matching parentheses using Stack

4. Technologies Used

- Python 3
- Visual Studio Code
- Command Prompt / Terminal
- Python Lists
- Linked Lists
- Linear Data Structures

5. Python Concepts Used

- Stack Data Structure

- Queue Data Structure
- LIFO Principle
- FIFO Principle
- Python Lists
- Linked Lists
- Circular Queue
- Push and Pop Operations
- Enqueue and Dequeue Operations
- Peek Operation
- Conditional Statements
- Loops
- User Input Handling
- Functions
- Data Structure Operations

6. Program Workflow

Start

↓

Implement Stack using Python List

↓

Perform Push, Pop and Peek Operations

↓

Implement Stack using Linked List

↓

Perform Stack Operations

↓

Implement Queue using Python List

↓

Perform Enqueue, Dequeue and Peek Operations

↓

Implement Circular Queue

↓

Check Empty and Full Conditions

↓

Check Balanced Parentheses using Stack

↓

Implement Task Scheduler using Queue

↓

Process Tasks using FIFO Order

↓

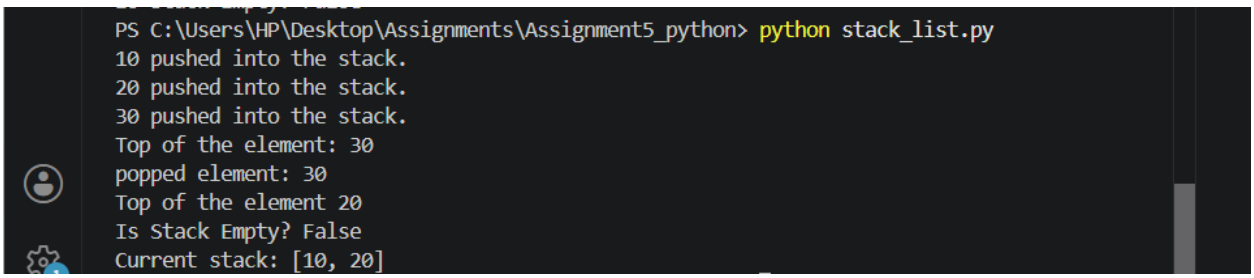
Display Results

↓

End

7. Output Screenshots and Explanation

Program 1: Stack using Python List



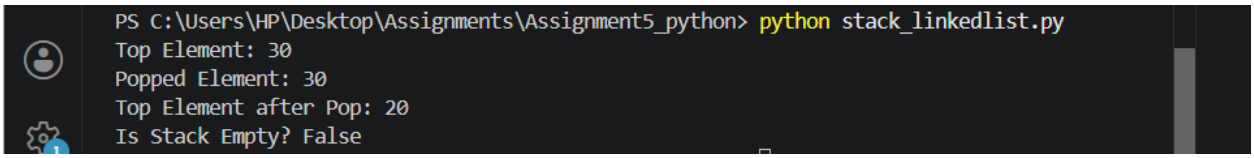
Explanation

The output shows the successful execution of Stack operations using a Python list. Elements are pushed onto the Stack, the top element is displayed, one element is removed using the pop operation, and the updated top element is displayed. Finally, the program checks whether the Stack is empty.

Key Observations

- Elements are inserted using the push() operation.
- The last inserted element appears at the top of the Stack.
- pop() removes the top element following the LIFO principle.
- peek() displays the current top element without removing it.

Program 2: Stack using Linked List



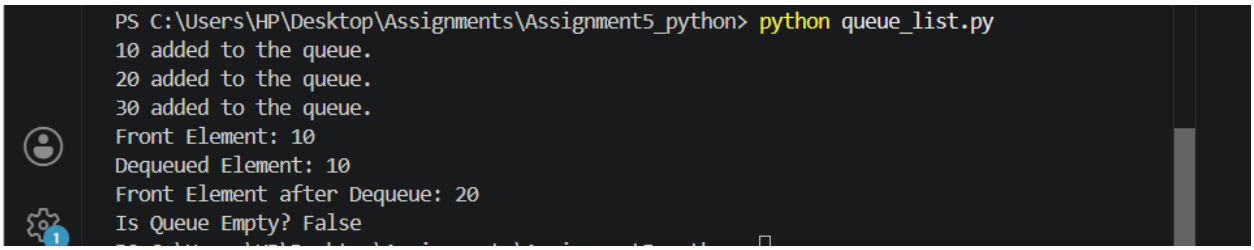
Explanation

The output demonstrates the implementation of a Stack using a linked list. Elements are added and removed from the top node, showing that Stack operations work correctly while maintaining the LIFO order.

Key Observations

- Stack is implemented using linked list nodes.
- New elements are added at the top.
- The top element is removed using pop().
- peek() displays the current top node.

Program 3: Queue using Python List



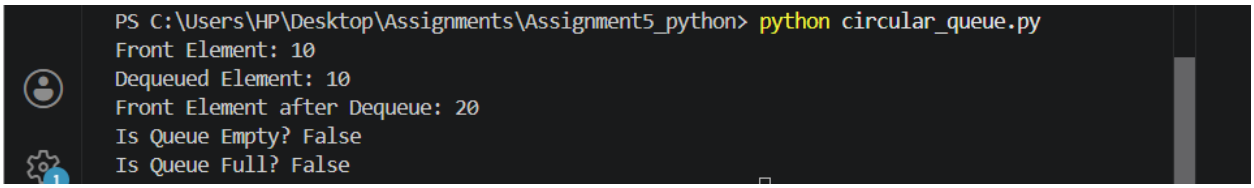
Explanation

The output shows the Queue operations performed using a Python list. Elements are inserted at the rear of the Queue and removed from the front, demonstrating the FIFO principle.

Key Observations

- Elements are added using enqueue().
- The first inserted element is removed first.
- peek() displays the front element.
- is_empty() checks whether the Queue contains elements.

Program 4: Circular Queue



Explanation

The output demonstrates the working of a Circular Queue using a fixed-size list. The front and rear pointers move circularly, allowing efficient utilization of available space.

Key Observations

- Elements are inserted using enqueue().
- Elements are removed using dequeue().
- Front and rear pointers move circularly.
- The Queue checks both empty and full conditions.

Program 5: Balanced Parentheses Checker

```
PS C:\Users\HP\Desktop\Assignments\Assignment5_python> python balanced_paranthesis.py
Enter input: ({]))
Not Balanced
```

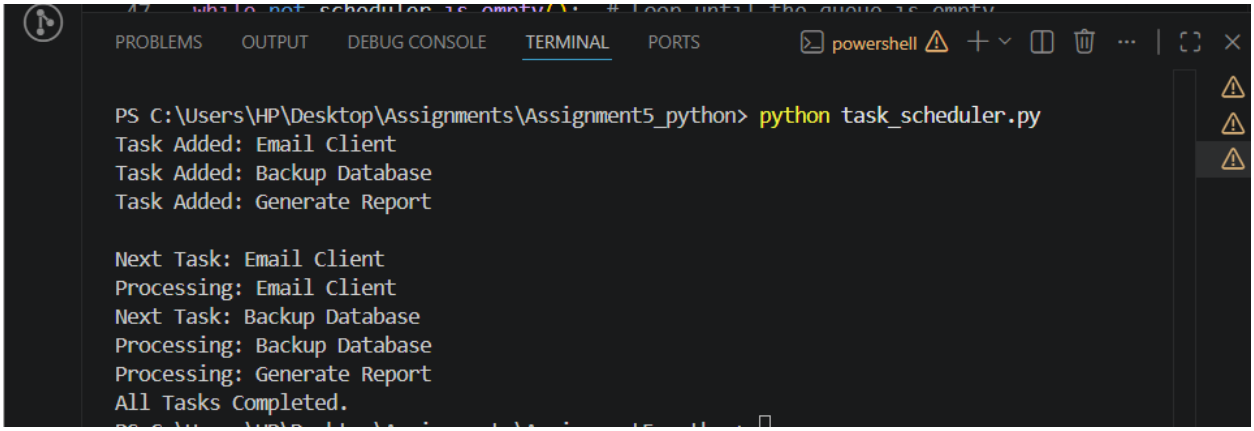
Explanation

The output verifies whether the given expression contains balanced parentheses. The program uses a Stack to compare opening and closing brackets and displays whether the expression is balanced.

Key Observations

- Opening brackets are pushed onto the Stack.
- Closing brackets are matched with the top element.
- Mismatched brackets are detected correctly.
- The final output indicates whether the expression is balanced.

Program 6: Task Scheduler



Explanation

The output demonstrates a simple task scheduler implemented using a Queue. Tasks are processed in the same order in which they are added, following the FIFO principle until all tasks are completed.

Key Observations

- Tasks are added to the scheduler Queue.

- The next task is displayed using peek().
- Tasks are processed one by one.
- A completion message is displayed after processing all tasks.

8. Learning Outcomes

After completing this project, I learned how to:

- Understand the fundamentals of linear data structures
- Implement Stack using different approaches
- Implement Queue using different approaches
- Apply the LIFO principle using Stack
- Apply the FIFO principle using Queue
- Understand Circular Queue operations
- Use Stack to solve balanced parentheses problems
- Use Queue to implement task scheduling
- Perform common operations such as push, pop, enqueue, dequeue, and peek
- Improve problem-solving skills through practical implementation of data structures

9. Conclusion

This project helped in understanding the implementation of Stack and Queue using different approaches in Python. It also demonstrated their practical applications through balanced parentheses checking and task scheduling, strengthening the understanding of linear data structures and their operations.